# ЛАБОРАТОРНА РОБОТА № 3

## Часткові представлення. Робота з View Components. Навігація. Фільтрація. Використання сесії.

***Мета:*** отримати навички роботи з частковими представленнями, навчитися працювати з параметрами рядку запиту, набути навичок роботи з сесіями.

**Хід роботи:**

**Завдання 1.** Створення часткового представлення (Partial View)

1. У папці Views/Shared створіть .cshtml файл для одиничного елементу зі списку сутностей (для SportsStore це буде ProductSummary.cshtml)
2. Перемістіть код для одиничного елементу з загального View Index.cshtm в створений файл.
3. Використайте створений Partial View в Index.cshtml за допомогою Tag Helper'а `<partial>`

## Листинг _SurveyListItem.cshtml

```
@model SurveyPortal.Models.Survey

<div class="col-md-6 mb-4">
    <div class="card shadow-sm">
        <div class="card-body">
            <h2 class="card-title">@Model.Title</h2>
            <p class="card-text">@Model.Description</p>
            <p><strong>Created:</strong> @Model.CreatedAt.ToString("yyyy-MM-dd HH:mm:ss")</p>
            <span class="badge bg-@(Model.IsActive ? "success" : "secondary")">
                @(Model.IsActive ? "Active" : "Over")
            </span>
            <br />
            <a asp-action="Details" asp-route-id="@Model.SurveyID" class="btn btn-outline-primary mt-
3">Details</a>
        </div>
    </div>
</div>
```

## Листинг Index.cshtml

```
@model SurveyPortal.Models.ViewModels.SurveyListViewModel

<div class="row">
    @foreach (var p in Model.Surveys)
    {
        <partial name="_SurveyListItem" model="p" />
    }
</div>
```

**Завдання 2.** Навігація по категоріям

Реалізуйте навігацію за категорією (або за іншою ознакою, яка більше підходить за вашим варіантом), дотримуючись наступних кроків:

1. В папці проекту створіть папку Components, в ній створіть файл з назвою NavigationMenuViewComponent.cs та реалізуйте його наступним чином:

```
using Microsoft.AspNetCore.Mvc;

namespace SportsStore.Components
{
   public class NavigationMenuViewComponent : ViewComponent
   {
      public string Invoke()
      {
         return "Hello from the Nav View Component";
      }
   }
}
```

**Листинг NavigationMenuViewComponent.cs після виконання завдання:**

```
using Microsoft.AspNetCore.Mvc;
using SurveyPortal.Models;

namespace SurveyPortal.Components
{
    public class NavigationMenuViewComponent : ViewComponent
    {
        private readonly SurveyDbContext _context;

        public NavigationMenuViewComponent(SurveyDbContext context)
        {
            _context = context;
        }
        public IViewComponentResult Invoke()
        {
            var categories = _context.Surveys
                .Select(s => s.Category)
                .Distinct()
                .OrderBy(c => c)
                .ToList();

            return View(categories);
        }
    }
}
```

2. В загальному шаблоні (файл _Layout.cshtml) додайте розмітку для сайд-бару. В доданій розмітці додайте виклик створеного NavigationMenuViewComponent. Структура вашої сторінки має бути схожою на ту, яка зображена на рисунку 2.1.
3. Змініть реалізацію NavigationMenuViewComponent таким чином, щоб компонент отримував перелік категорій з БД і повертав результат типу IViewComponentResult.

**Листинг _Lauout.cshtml після виконання завдання:**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"]SurveyPortal</title>
    <link rel="stylesheet" href="~/lib/bootstrap/css/bootstrap.min.css" />
```

```
    <link rel="stylesheet" href="~/css/site.css" />
</head>
<body class="d-flex flex-column min-vh-100">

    <header class="bg-dark text-white py-3">
        <div class="container">
            <h1 class="mb-0">Survey Portal</h1>
        </div>
    </header>

    <main class="flex-grow-1 container mt-4">
        <div class="row">
            <main class="flex-grow-1 container mt-4">
                <div class="row">
                    <div class="col-md-3">
                        @await Component.InvokeAsync("NavigationMenu")
                    </div>
                    <div class="col-md-9">
                        @RenderBody()
                    </div>
                </div>
            </main>

        </div>
    </main>

    <footer class="bg-dark text-white text-center py-3 mt-auto">
        <p>&copy; @DateTime.Now.Year - Survey Portal</p>
    </footer>

    <script src="~/lib/bootstrap/js/bootstrap.bundle.min.js"></script>

    @RenderSection("Scripts", required: false);
</body>
</html>
```

4. В папці Views/Shared/Components/NavigationMenu створіть файл Default.cshtml.
5. В створеному файлі представлення реалізуйте виведення списку посилань, отриманих з ViewComponent'а.

**Листинг Default.cshtml після виконання завдання:**

```
@model SurveyPortal.Models.Survey

@{
  var sessionKey = $"draft_{Model.SurveyID}";
  var draftAnswer = Context.Session.GetString(sessionKey) ?? "";
}

@section Scripts {
  <script>
    const textarea = document.getElementById("surveyAnswer");
    const surveyId = "@Model.SurveyID";
    let timeout;

    textarea.addEventListener("input", function () {
      clearTimeout(timeout);

      timeout = setTimeout(() => {
        fetch(`/Home/SaveDraftAnswer?surveyId=${surveyId}`, {
          method: "POST",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({ answer: textarea.value })
        });
      }, 500);
    });
  </script>
}

<div class="container mt-4">
  <h2>@Model.Title</h2>
  <p class="text-muted">@Model.Description</p>
  <p><strong>Created:</strong> @Model.CreatedAt.ToString("yyyy-MM-dd HH:mm:ss")</p>
  <span class="badge bg-@(Model.IsActive ? "success" : "secondary")">
```

```
    @(Model.IsActive ? "Active" : "Over")
  </span>

  @if (Model.IsActive)
  {
    <form asp-action="SaveSurveyAnswer" method="post" class="mt-4">
      <input type="hidden" name="SurveyID" value="@Model.SurveyID" />

      <div class="mb-3">
        <label for="surveyAnswer" class="form-label">Your Answer:</label>
        <textarea id="surveyAnswer" name="Answer" class="form-control" rows="6" placeholder="Type your answer here..."
required>@draftAnswer</textarea>
      </div>

      <button type="submit" class="btn btn-primary">Submit</button>
    </form>
  }
  else
  {
    <p class="mt-3 text-muted">This survey is no longer accepting responses.</p>
  }

  @if (Model.SurveyAnswers.Any())
  {
    <hr />
    <h5 class="mt-4">Previous Answers:</h5>
    <ul class="list-group">
      @foreach (var answer in Model.SurveyAnswers.OrderByDescending(a => a.AnsweredAt))
      {
        <li class="list-group-item">
          <small class="text-muted">@answer.AnsweredAt.ToString("yyyy-MM-dd HH:mm")</small><br />
          @answer.Answer
        </li>
      }
    </ul>
  }
</div>
```

6. У файлі Controllers/HomeController.cs в методі Index() реалізуйте підтримку фільтрації по категоріям.
7. Змініть ViewModel таким чином, щоб мати інформацію про обрану категорію на представленні. Це знадобиться для формування посилань на сторінки з урахуванням обраної категорії.
8. Реалізуйте виділення категорії, обраної користувачем. Для цього передавайте інформацію про обрану категорію з View Component в View.
9. Змініть реалізацію пейджингу таким чином, щоб він відображав кількість доступних сторінок відповідно до обраної категорії.

**Листинг HomeController.cs після виконання лабораторної:**

```
using Microsoft.AspNetCore.Mvc;
using SurveyPortal.Models;
using SurveyPortal.Models.ViewModels;

namespace SurveyPortal.Controllers
{
    public class HomeController : Controller
    {
        private ISurveyRepository _repository;
        public const int PageSize = 4;

        public HomeController(ISurveyRepository repository)
        {
            _repository = repository;
        }
        public IActionResult Index(string? category, int page = 1)
        {
            var query = _repository.Surveys;

            if (!string.IsNullOrEmpty(category))
            {
                query = query.Where(s => s.Category == category);
            }
```

```
            var model = new SurveyListViewModel
            {
                Surveys = query
                .OrderBy(s => s.CreatedAt)
                .Skip((page - 1) * PageSize)
                .Take(PageSize),
                PagingInfo = new PagingInfo
                {
                    CurrentPage = page,
                    ItemsPerPage = PageSize,
                    TotalItems = query.Count()
                },
                CurrentCategory = category
            };

            return View(model);
        }
        public IActionResult Details(long id)
        {
            var survey = _repository.Surveys.FirstOrDefault(s => s.SurveyID == id);

            if (survey == null)
            {
                return NotFound();
            }

            return View(survey);
        }
        [HttpPost]
        public IActionResult SaveSurveyAnswer(long SurveyID, string Answer)
        {
            var survey = _repository.Surveys.FirstOrDefault(s => s.SurveyID == SurveyID);

            if (survey == null || string.IsNullOrWhiteSpace(Answer))
            {
                return NotFound();
            }

            _repository.AddAnswer(new SurveyAnswers
            {
                SurveyID = SurveyID,
                Answer = Answer,
                AnsweredAt = DateTime.UtcNow
            });

            TempData["Message"] = "Thank you for your answer!";
            return RedirectToAction("Details", new { id = SurveyID });
        }

        public IActionResult SaveDraftAnswer(long surveyId, [FromBody] DraftDto dto)
        {
            var key = $"draft_{surveyId}";
            HttpContext.Session.SetString(key, dto.Answer ?? "");

            return Ok();
        }

        public class DraftDto
        {
            public string Answer { get; set; }
        }
    }

}
```

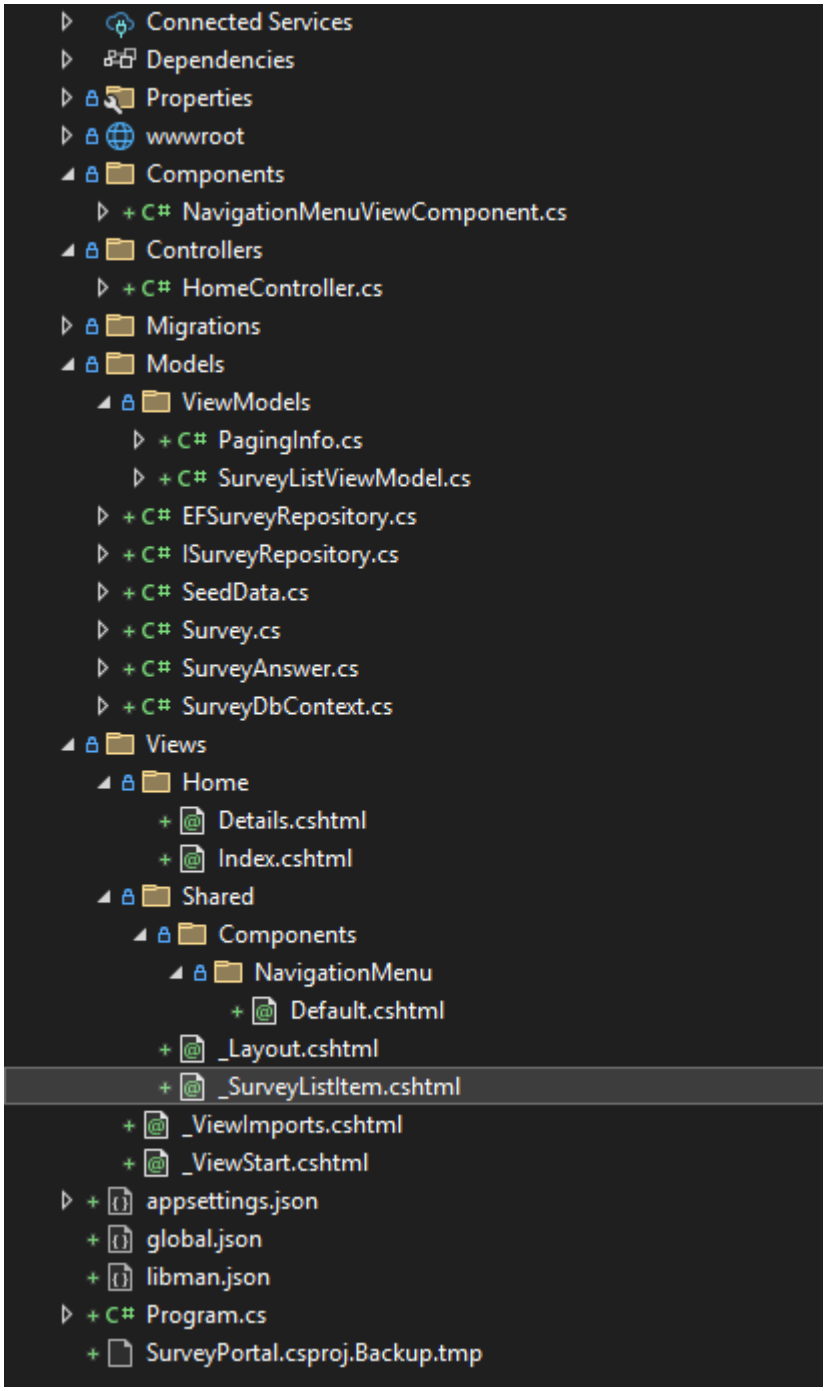


Рис. 1. Структура проекту після виконання лабораторної роботи.

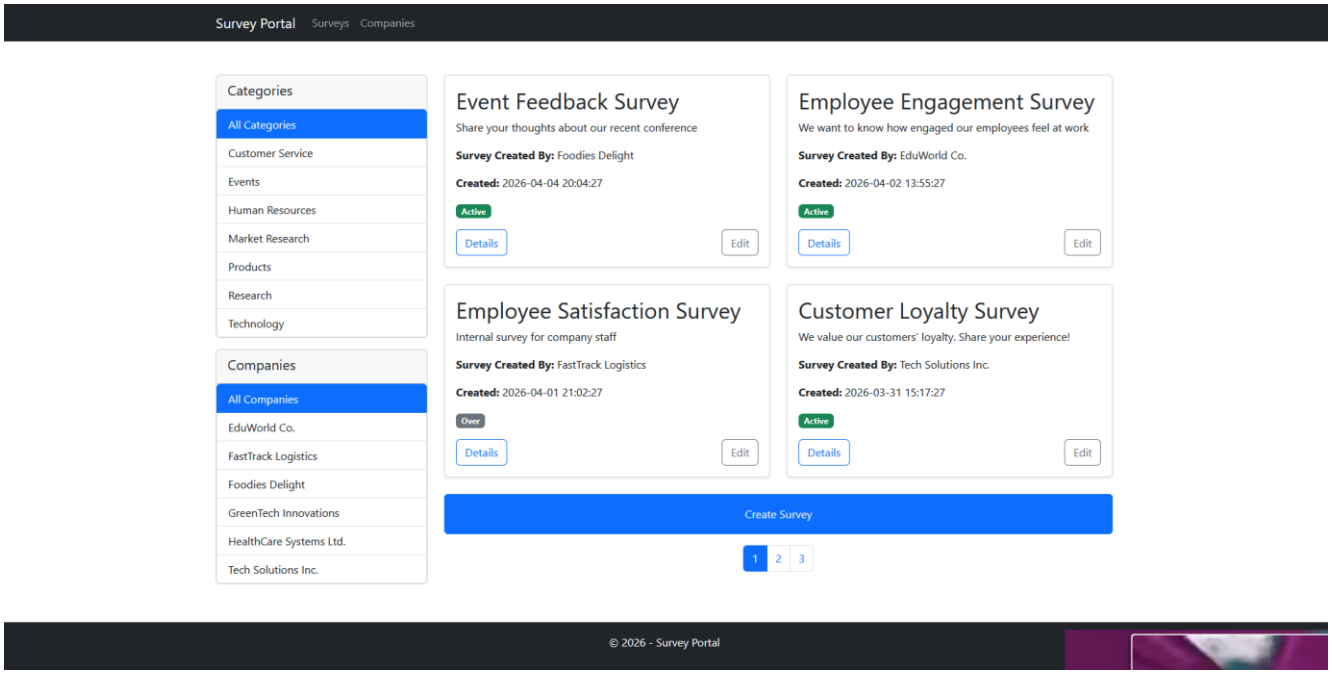


Рис. 2. Вигляд сайту після виконання лабораторної роботи.

## Завдання 3. Сесії

Реалізуйте функціональність відповідно до варіанту вашого проекту з використанням механізму сесії.

| | | |
|---|---|---|
| 5 | Портал для опитувань | Збереження незавершених відповідей на опитування у сесії, щоб користувач міг завершити опитування пізніше. |

Рис. 3. Варіант.

## Листинг Details.cshtml:

```
@model SurveyPortal.Models.Survey

@{
    var sessionKey = $"draft_{Model.SurveyID}";
    var draftAnswer = Context.Session.GetString(sessionKey) ?? "";
}

@section Scripts {
    <script>
        const textarea = document.getElementById("surveyAnswer");
        const surveyId = "@Model.SurveyID";
        let timeout;

        textarea.addEventListener("input", function () {
            clearTimeout(timeout);

            timeout = setTimeout(() => {
                fetch(`/Home/SaveDraftAnswer?surveyId=${surveyId}`, {
                    method: "POST",
                    headers: {
                        "Content-Type": "application/json"
                    },
                    body: JSON.stringify({ answer: textarea.value })
                });
            }, 500);
        });
    </script>
}

<div class="container mt-4">
    <h2>@Model.Title</h2>
```

```
    <p class="text-muted">@Model.Description</p>
    <p><strong>Created:</strong> @Model.CreatedAt.ToString("yyyy-MM-dd HH:mm:ss")</p>
    <span class="badge bg-@(Model.IsActive ? "success" : "secondary")">
        @(Model.IsActive ? "Active" : "Over")
    </span>

    @if (Model.IsActive)
    {
        <form asp-action="SaveSurveyAnswer" method="post" class="mt-4">
            <input type="hidden" name="SurveyID" value="@Model.SurveyID" />

            <div class="mb-3">
                <label for="surveyAnswer" class="form-label">Your Answer:</label>
                <textarea id="surveyAnswer" name="Answer" class="form-control" rows="6" placeholder="Type your
answer here..." required>@draftAnswer</textarea>
            </div>

            <button type="submit" class="btn btn-primary">Submit</button>
        </form>
    }
    else
    {
        <p class="mt-3 text-muted">This survey is no longer accepting responses.</p>
    }

    @if (Model.SurveyAnswers.Any())
    {
        <hr />
        <h5 class="mt-4">Previous Answers:</h5>
        <ul class="list-group">
            @foreach (var answer in Model.SurveyAnswers.OrderByDescending(a => a.AnsweredAt))
            {
                <li class="list-group-item">
                    <small class="text-muted">@answer.AnsweredAt.ToString("yyyy-MM-dd HH:mm")</small><br />
                    @answer.Answer
                </li>
            }
        </ul>
    }
</div>
```

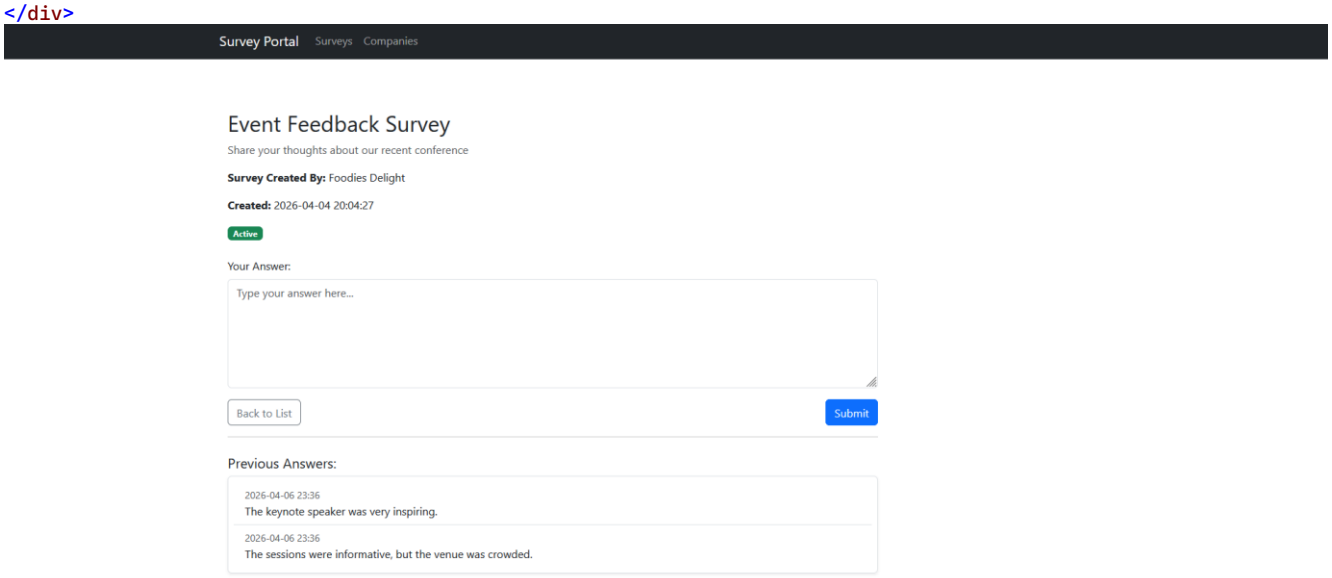


Рис. 4. Details.

Задля реалізації сесії в Program.cs були внесені зміни.

## Листинг Program.cs:

```
using Microsoft.EntityFrameworkCore;
using SurveyPortal.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();
```

```
builder.Services.AddDistributedMemoryCache();

builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromSeconds(10);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});

builder.Services.AddDbContext<SurveyDbContext>(opts =>
{
    var connectionString = builder.Configuration.GetConnectionString("SurveyPortalConnection");
    opts.UseMySql(connectionString, ServerVersion.AutoDetect(connectionString));
});

builder.Services.AddScoped<ISurveyRepository, EFSurveyRepository>();

var app = builder.Build();

app.UseStaticFiles();

app.UseSession();

app.MapDefaultControllerRoute();

app.MapControllerRoute
    (
    name: "pagination",
    pattern: "Home/Index/{page:int}",
    defaults: new { Controller = "Home", action = "Index" }
    );

SeedData.EnsurePopulated(app);

app.Run();
```

Посилання на репозиторій: https://github.com/JanRizhenko/ASP.NET-course

***Висновок:*** Вивчення роботи з частковими представленнями, View Components, навігацією та фільтрацією дозволяє ефективно організовувати структуру веб-додатків, забезпечуючи модульність та гнучкість. Використання параметрів рядка запиту дозволяє інтерактивно керувати даними та фільтрацією контенту, а сесія забезпечує збереження інформації між запитами, що важливо для збереження незавершених дій користувача. Ці навички дозволяють створювати більш динамічні та зручні веб-додатки.